THE LAB — DAY 05

The Digit Reader

Superpowers: PyTorch + GPU

Thursday · July 23, 2026 **Unlocks ✍️ Digit Reader**

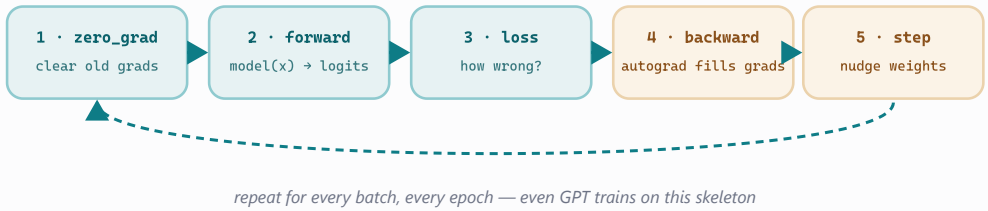
1 Mission briefing

Yesterday you hand-built a network's engine in pure NumPy — and felt the pain: ~130 forward passes just to nudge the weights *once*. Today the Lab issues power tools. **PyTorch** computes those nudges automatically and exactly, and a **GPU** runs them at ludicrous speed. *Previously:* you earned a deep respect for backpropagation — and calluses. Your mission: teach a network to read handwriting, then point it at *your own*.

- ☐ Meet **tensors** — arrays that live on a GPU and track history
- ☐ Let **autograd** compute gradients for you
- ☐ Train a real network on **MNIST** handwriting
- ☐ Watch a **GPU** demolish a CPU at matrix multiply
- ☐ Upgrade to a **convolutional** network (CNN)
- ☐ Draw your own digit and **unlock the Digit Reader**

2 Field guide the concepts

Tensors and autograd. A **tensor** is a NumPy array with two superpowers: it can live on a **GPU**, and it remembers its own history. Mark it `requires_grad=True`, do some math, call `.backward()`, and PyTorch replays the chain rule to leave the exact gradient in `.grad` — the same job yesterday's nudge method did, but in **two passes instead of 130**. Almost every network in the world then trains with the **same five-step loop**. Burn it into memory:



The five lines that train (almost) all of AI. Under the hood it's all the `@` matrix multiply from Day 4.

MNIST, CNNs, and the GPU. You'll train on **MNIST** — 70,000 handwritten digits — in **batches**, for a few **epochs** (full passes), with the **Adam** optimizer. A plain MLP flattens the image; a **convolutional** network keeps its 2-D shape and scores higher. And because every step is matrix multiplication, a **GPU** that does thousands of multiplies at once leaves the CPU in the dust.

3 Lab missions check each off in the notebook

1 Your first tensors ☐

Matrix-multiply a 3×4 and a 4×2 tensor with `@`.

Expect: `C` is 3×2 — the shared inner dimension cancels.

2 Differentiate by yourself (well, by PyTorch) ☐

Build `y = x**3 + 2*x` at x = 3, call `y.backward()`.

Expect: `x.grad` is 29 ($3x^2+2$ at x=3).

3 Assemble the MLP ☐

`nn.Sequential`: Flatten → Linear(784,128) → ReLU → Linear(128,10).

Expect: a four-layer model, moved onto `DEVICE`.

4 Count the parameters ☐

Sum `p.numel()` over `mlp.parameters()`.

Expect: exactly 101,770 tunable knobs.

5 Complete the training loop ☐

Fill the five gaps: zero_grad → forward → loss → backward → step.

Expect: the printed loss goes down each epoch.

6 Write `evaluate` ☐

A `torch.no_grad()` loop counting correct `argmax` predictions.

Expect: MLP test accuracy ≥ 0.95 (~97%).

7 Time the matmul ☐

Time `A @ B` on CPU vs GPU (with `cuda.synchronize()`).

Expect: the GPU obliterates the CPU on big matrices.

8 Train the CNN by reusing your loop ☐

Instantiate `SmallCNN` and call the same `train_model`.

Expect: accuracy climbs a couple of points above the MLP.

4 Cheat sheet PyTorch essentials

`torch.from_numpy(a)` · `t.numpy()`	Bridge between NumPy and torch
`x = torch.tensor(2., requires_grad=True)`	Track history so autograd can differentiate
`y.backward(); x.grad`	Fill in every gradient in one backward pass
`tensor.to(DEVICE)`	Move data / model onto the GPU (or CPU)
`nn.Sequential(nn.Flatten(), nn.Linear…, nn.ReLU…)`	Stack layers front to back
`nn.CrossEntropyLoss()` · `optim.Adam(…, lr=1e-3)`	Classification loss · the optimizer
`zero_grad → model(x) → loss → backward → step`	The five-line training loop
`with torch.no_grad(): logits.argmax(dim=1)`	Evaluate without tracking gradients
`DataLoader(ds, batch_size=128, shuffle=True)`	Feed data in shuffled batches
`torch.jit.trace(model, ex).save("….pt")`	Export a frozen TorchScript model

5 🔓 Unlock your Studio module

✍️ Digit Reader

Export the trained CNN as a self-contained TorchScript file the Studio can load without your training code. It must write exactly:

`ai_studio/models/digit_reader.pt`

Input contract: a `(1, 1, 28, 28)` tensor, values 0–1, **white digit on black** (MNIST style — `ToTensor` only, no normalization). Then:

```
$ python ai_studio/app.py
```

6 Mini-glossary

tensor
A NumPy-like array that can run on a GPU and track its own history.

autograd
PyTorch's automatic gradients via one backward pass of the chain rule.

epoch
One full pass over the whole training set.

batch
A small group of examples processed together in one step.

optimizer
The rule (e.g. Adam) that turns gradients into weight updates.

CUDA / GPU
Hardware that does thousands of multiplies in parallel.

TorchScript
A frozen, portable copy of a model, saved as a `.pt` file.

7 Tonight's show & tell

1. What test accuracy did your **MLP** reach — and how many more digits did the **CNN** get right?
2. Draw three digits in your Studio's **Digit Reader**. Which one fooled it, and why do you think it slipped?
3. In the GPU race, how many times faster was the biggest **matmul**? Tie that back to Weeks 3–4.